

编程能力与 Debug 能力说明

申请实验室日常科研实习 · 以 I3DM 复现项目作答

考核原题 (问题 2)

因为编程能力是做好计算机科研的基础，实验室需要考察申请者的编程能力。请简单描述一个最能体现自己编程能力和 Debug 能力的项目，并说明：① 哪些部分体现编程能力；② 哪些部分体现 Debug 能力；③ 一般的 Debug 思路是怎样的。

1. 选用的项目：I3DM 复现 (科研向工程闭环)

我选的项目是：在科大本科生算力平台 (单卡 A100、单作业 4 小时配额) 上，复现长视频 3D 一致生成方法 I3DM。它直接对应实验室看重的能力——不是交一个能演示的网页，而是在真实约束下把数据 → 评测 → 训练冒烟收成可重复的科研工程闭环。

具体完成：① 推理评测——Re10K 上 32 个场景全部产出生成/真值对照漫游视频；② 冒烟训练——LoRA 200 步跑通，checkpoint 落盘，验证前向/反向数值健康。未重写网络结构；完整 11000 步训练经评估放弃；官方 eval 不聚合 FVD/PSNR，本轮以管道正确性与定性一致性为验收标准。

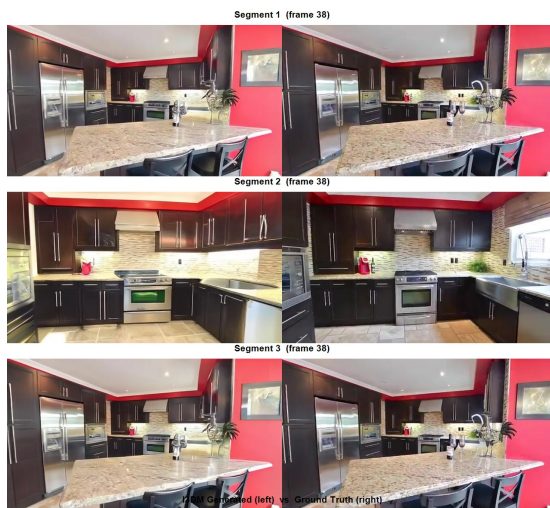


图 1 场景 005dd9a58df1ba3c: 左列生成 / 右列真值 (第 1/3/6 段中间帧)。单张参考图驱动约 28 秒自回归漫游——推理管道跑通的实证。

2. 哪些部分体现编程能力

编程能力体现在：能读懂他人训练/评测代码，在集群配额下把接口与流程写成可复现、可接力的脚本——这是进组后做科研实验的日常基本功。

(1) 数据准备与校验

按场景索引按需下载 Re10K (相对全量约省 78%)，解包后切出 530 个 77 帧训练片段，并对样本张量形状预检，保证训练入口格式正确。对应「训练数据准备」能力。

(2) 评测工程与断点续跑

单场景约 13–14 分钟，4 小时装不下大批量任务。场景列表分块提交；评测脚本增加：已有 video_combined.mp4 则跳过。最终 32/32 零失败产出。

(3) 训练管线接入与版本锁定

锁定 torch 2.5.0+cu124 / xformers / transformers 组合，配置 DiT LoRA 与 ref_conv、nvs_model，用 sbatch + accelerate 提交可重复的冒烟训练。

(4) 约束下的取舍

配额是方案的一部分。完整训练需大量

4

小时接力，性价比低；改为

200

步冒烟验证管道——能判断「什么必须做、什么可以不做」。

关键代码逻辑

评测续跑: if exists(video_combined.mp4): skip

训练配置: LoRA rank=1024 · trainable=ref_conv,nvs_model · save_steps=50 · wandb offline

3. 哪些部分体现 Debug 能力

Debug 能力体现在：失败可复现、假说可检验、错误原因能被对照实验否定。训练冒烟阶段连续五次提交，其中 OOM 定位是最能说明问题的一段。

(1) 冒烟训练排错表（核心实例）

作业	现象	动作与结论
52883	缺 pandas	补装 2.3.3
53053	缺 easydict	扫描 import，一次性补齐
53054	缺 wandb → CUDA OOM (77 帧前向 ≈78.5 GiB)	补装后开始显存剖析
53055	rank 1024→512 仍 OOM (显存几乎不变)	否定「LoRA 过大」： 瓶颈是帧数激活
53057 □	num_target_views 77→41 (满足 VAE 时间维整除)	rank 仍 1024，一次通过 200 步 / 42 分钟

表 1 关键转折：降 LoRA rank 不能降显存 → 证明激活由帧数驱动 → 改目标帧数为 41。

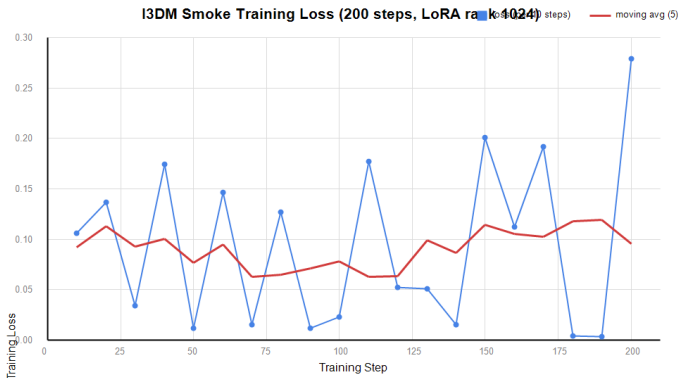


图 2 200 步 loss（蓝：原始；红：滑动平均）。扩散单步 loss 天然高噪声；验收标准是无 NaN、梯度稳定、checkpoint 可存——不是要求短训出现论文级平滑下降。

(2) 其他典型 Debug

- 路径：数据放登录节点 /tmp，计算节点不可见 → 改共享主目录。
- 依赖：官方 requirements 不宜全量硬装；隐式依赖爆雷后改为提交前扫描 import。
- 调度：4 小时超时 → 分块 + 续跑，而不是硬扛。

4. 一般的 Debug 思路

我把 Debug 当成小型科学实验：先复现，再假说，用最小对照检验；假说被否定就换方向。这与系统化调试中「观察 → 假说 → 预测 → 实验 → 修正」一致。结合 I3DM 的 OOM：

- ① 稳定复现
- 收成最小可重复案例（单场景评测、短步数训练）。复现不了，不急着改。

② 分层隔离

环境依赖 / 数据路径 / 前向与显存 / 作业调度。每次只动一层。

③ 假说 + 对照

写下「若原因是 X，改 X 后应如何」。OOM 时假说「LoRA 过大」；预测降 rank 显存下降；实验几乎不变 → 否定 → 转向帧数。

④ 最小修改 + 冒烟

只改必要处（帧数与 VAE 整除），先用 200 步确认，再放大。

⑤ 回归与留痕

确认问题消失；记下有效配置与失败作业号，避免重复踩坑。

一句话：Debug 的目标不是「改到能跑」，而是「用对照实验证明找对了原因」。降 rank 那一步的价值不在于成功——而在于失败得干净，把真因锁到帧数。这正是我希望带进实验室、用来独立推进科研实验的基本能力。

附图与数据：results_showcase/ · 完整 32 场景结果存于算力平台 · 仅作问题 2 作答材料